CS 4100: Artificial Intelligence (Fall 2026) – Final Project

1 General Information

The project offers an opportunity to apply concepts/techniques learned in class on a substantial problem that interests students. Whereas most of the course has been focused on breadth of topics, the project provides a counterpoint by requiring students to study a problem in depth.

Topics

Your final project can be on anything that has some relation to AI. It should relate to at least one of the three modules we cover in our class (search, MDPs / RL, machine learning / Bayesian inference). It is likely that the project you choose only relates to one module – this is completely fine and expected. We suggest many topics in a later section of this document.

Types

This semester, we offer two types of projects:

- **Practical**: Find and formulate a problem, develop/implement/apply algorithms to solve it. Recommended if you want to escape the toy domains we have studied so far and actually build something substantial and potentially applicable in the real world. You would likely learn how to use new AI-related software tools and hone algorithmic / data-scientific skills. If successful, the project would be a great addition to your portfolio. Also a great chance to do the AI-related hobby project that you have always dreamed of but never got started on.
- **Theoretical**: Do an independent study of an advanced topic not covered in the course. Recommended if you have enjoyed the topics so far and just want to see more of it, or go significantly deeper into something we briefly touched on. You will learn about new problems / models / algorithms. This is recommended if you plan to take more advanced AI courses in the future (e.g., machine learning), and/or if you are interested in getting involved in AI-related research. Could also be used to deeply study / reproduce a research paper.

Teams

Teams should have 1–3 members. Our recommendation:

- Practical project: We strongly recommend forming teams of 2, which in past experience has led to much more interesting projects. Overall, we expect each team member to spend about 40–60 hours on the project (about 8-10 hours per week). This is why teams of two are recommended – you can accomplish much more with 100 total hours compared to 50.
- Theoretical project: Teams of 1 or 2 are recommended. If in a team of 2, the chosen topic should be broader, and there should be a rationale as to why 2 people are needed.

Deviating from the above recommendations will require a written rationale in the proposal, and potentially approval from / further discussion with Prof. Wong. Teams with more members (e.g., 3 for practical or 2 for theoretical) will be expected to produce more substantial projects.

Expectations

We are mostly interested in the **process** of your project, and not the final results.

- Practical project: We do not expect everyone to achieve state-of-the-art results on the problem they choose – although that would be a pleasant surprise if it happened. Your team is expected to find a problem of interest, formulate it well and precisely, develop/apply an algorithm to solve the problem, analyze your results, and communicate that in a report.
- Theoretical project: You should learn the proposed topic and understand it well enough to the point that you can potentially teach a lecture on it. The default expectation is that you will deliver a 20–30 minute recorded video lecture on your topic, which will be made available to current and future generations of CS 4100 students. Some small-scale implementation of relevant algorithm(s) is also expected, primarily as pedagogical examples to supplement your lecture. However, the exact deliverables can be discussed with course staff.

Timeline

All deadlines, except for the project presentation, are at **11:59 PM ET**.
Submissions should be uploaded to the respective assignments on **Canvas**.

- The project pre-proposal form is due 10/30 (Fri).
- The project proposal is due 11/6 (Fri).
- There is one self-proposed halfway milestone, which will be handed in on 12/4 (Fri).
- The draft project report (practical) / deliverables (theoretical) is due on 12/11 (Fri).
- **The final project report / deliverables is due on 12/18 (Fri), 11:59 PM ET.**
  This is rather strict, because final grades are due soon after that, and we need time to ensure every project receives a proper assessment.

Page 9 contains further details and expectations on good project progress.

Typical project process (practical track)

Overall, we expect each team member to spend about 40–60 hours on the project (about 8-10 hours per week). This is why teams of two are recommended – you can accomplish much more with 100 total hours compared to 50.

1. ($\sim$ 5 hours) Decide on a project topic, do some background reading, and make an initial problem formulation. Make a rough plan for the project itself (this is the proposal).
2. ($\sim$ 10 hours) Learn the additional knowledge necessary to undertake the project. The amount of time this takes may vary depending on your chosen topic. It is very likely that you will need to learn at least one or two new concepts/algorithms for your project.
3. ($\sim$ 10 hours) Figure out the simplest way to solve your problem (i.e., a basic algorithm), and implement it / use an existing implementation. Apply the simple algorithm to your problem.
4. ($\sim$ 5 hours) There are usually two results of step 3: Either you have completely solved the problem you chose, or it doesn't work too well (at least on some cases). If the former, analyze why it did so well, and come up with some interesting extensions. If the latter, analyze what went wrong, and figure out how to address that.
5. (Remaining time: $\sim$ 10–30 hours?) Iterate steps 3-4, each time using the analysis in step 4 to identify some potential area to improve on, and try again (step 3).
6. Summarize the overall process: the problem, algorithm, results, and insightful analyses. Present your story to the class / in your final report.

Typical project process (theoretical track)

This is more fluid and harder to predict. It depends on the chosen topic and its scope.

1. ($\sim$ 20–30 hours) Read the textbook and/or other tutorial material, possibly including watching recorded lectures, to gain a broad understanding of the topic and its context.
2. ($\sim$ 10–20 hours) Work on problems / code simple examples and experiment with variations, to solidify and deepen the initial understanding, and to sort out confusions and misconceptions.
3. ($\sim$ 10–20 hours) Distill your understanding into project deliverables (such as in the form of a lecture video, report, slides, illustrative examples, or other pedagogical materials).

Throughout the project, there may be more frequent check-ins with course staff to ensure that your independent study is on the right track, and to jointly define what the final results of the project may look like.

2 Potential Topics

This section provides some general topic categories and specific suggestions, but you are welcome to choose anything else that interests you.

Topic categories

- (Practical) Apply search / MDPs / RL to your favorite single-player / multi-player game. About half of the projects we see fall into this category.
- (Practical) Extend a programming assignment (Pacman) in much greater depth. For example, you can try to apply and analyze even better heuristics / search algorithms (potentially learning parts of the heuristic / algorithm), learn better evaluation functions (learn the features and weights), or try out better reinforcement learning algorithms (potentially with learned features and weights for function approximation).
- (Practical) Find some interesting problem / dataset on Kaggle or some other competition platform and tackle it. This is usually a machine learning problem. If you do well, you might be able to enter/win a competition!
- (Practical) Gymnasium – This platform (originally OpenAI Gym) contains many interesting environments for training reinforcement learning (RL) algorithms. It is quite popular for RL enthusiasts and is often used to benchmark RL algorithms. There is an increasing number of similar platforms.
- (Theoretical) Learn extensively about an problem / model / algorithm that is just beyond the scope of the course. Develop / apply and convey the understanding on illustrative examples. Produce pedagogical materials about the topic. See below for some suggested specific topics.
- (Theoretical) Find a paper that tackles an interesting problem, and try to re-implement it (initially without referring to existing code, if any is available). This can be quite challenging because you may find that some crucial details (e.g., particular settings of constants or hyperparameters) may not be present in the paper!
- (Theoretical) If you have ideas about how to substantially improve and liven pedagogical materials used in current class topics, and want to improve the learning experience of future students, you can implement them in a project. Substantial improvement means the introduction of illustrative examples, visualizations, interactive demonstrations, or other engaging material. This will require frequent discussion with Prof. Wong.

Potential problems / models / algorithms (theoretical track)

- Theoretical correctness / convergence / optimality proofs of algorithms covered in class (e.g., A*, value iteration, policy iteration, Q-learning, etc.)
- Alpha-beta pruning and other search-pruning strategies (AIMA Ch. 5.3)
- Constraint satisfaction problems (AIMA Ch. 6)
- Monte-Carlo tree search (AIMA Ch. 5.4, briefly seen in Lecture 7)
- Local search (AIMA Ch. 4, Ch. 6.4)
- Classical planning (AIMA Ch. 10-11)
- Policy iteration, MDP solution methods (AIMA Ch. 17.2, Sutton and Barto Ch. 4)
- Bandit problems (AIMA Ch. 17.3, Sutton and Barto Ch. 2)
- Partially observable Markov decision processes (POMDPs, AIMA Ch. 17.4)
- Multi-agent decision making (AIMA Ch. 18)
- Policy search / policy gradient (AIMA Ch. 22.5, Sutton and Barto Ch. 13)
- Kalman filters / dynamic Bayesian networks (AIMA Ch. 14.4-14.5)
- Approximate inference algorithm, e.g., Markov-chain Monte Carlo (MCMC) (AIMA Ch. 13.4)
- Learning Bayesian networks / expectation maximization (AIMA Ch. 20)
- Variations on stochastic gradient descent algorithms
- Your favorite machine learning problem / algorithm (AIMA Ch. 19, Ch. 21; also see recent offerings of CS 6140)

As you can see, many of the above suggestions are actually covered in AIMA, but we do not have time to cover them. Flip through the textbook and see what interests you. If you choose such a problem / algorithm though, expect to read further beyond the content in the textbook – you will most likely need more depth than the textbook covers.

Projects that are *not* recommended

- Apply an existing algorithm to an existing dataset and report the results. This is not enough – you need to analyze the results, see what was good and what could be improved, and iterate.
- Projects on unrelated topics – if you are unsure, ask us!
- Projects that are too broad. 40-60 hours (or 80-120 for teams of 2) is not a lot of time. Start small (very small), and if you succeed early, extend and iterate from there. If there is an interesting problem that is likely to take too much time (this is true for many interesting problems), identify the first step in the problem and make that your project.

Comments regarding implementation (code)

- **All projects should have to have an implementation component**, including theoretical projects. For practical projects, implementation will likely be the majority of the work. For theoretical projects, the implementation will likely be limited to toy/illustrative domains – for example, if your theoretical project was on Monte-Carlo tree search, then you may be implementing the algorithm on some simple search problems to demonstrate the algorithm.
- If there is an existing implementation, you can use it (with proper acknowledgment), but you do not have to. It depends on what you want to get out of the project. If you want to extend an existing method, then you will probably save time by building on existing implementations (especially if this is some non-AI related infrastructure, e.g., a game engine). Likewise, you are welcome to AI coding tools to implement non-AI related infrastructure, e.g., a GUI.
- If you want to have the experience of writing from scratch and deeply understanding the existing approach, you are welcome to do a re-implementation (although you should not refer to the existing "answer" in this case in your initial attempt). If you choose to re-implement, you may find that you cannot replicate existing performance, at which point you can compare against the existing implementation and see what went wrong – is it a bug on your end, or are there some 'magic numbers' that make the algorithm work? Analyzing these hidden / unexpected bits can make a very interesting project.

Additional advice

The most common mistake is setting the project scope to be too large.
It is very easy to define AI problems that are computationally intractable. The only way to succeed is to define a set of increasingly challenging sub-problems, starting from ones that are so simple that the solution should be obvious (both to you and to the AI algorithm).

- If your problem has partial observability, try defining a fully observable version.
- If your problem has multiple agents/players, try defining a two-player / one-player version.
- If your problem has a long horizon, try solving a shorter horizon problem (e.g., the "endgame").
- If you are training a large model on lots of data, try using a smaller model with less data.
- Come up with sanity-check examples (similar to unit tests) where you are certain what the correct answer/solution is, and make sure your AI algorithm can solve those too.
- The problems you use to develop your method on should be very fast to run on, ideally in seconds, and certainly under a minute (or perhaps 10 minutes in later stages). If you need to run your method overnight after every change, that is far too slow to iterate with.

3 Project Proposal

Each team submits one proposal. The project proposal is a short document that organizes your team's intentions and communicates that to the course staff, such that we can provide appropriate guidance. It is not a contract – we expect that projects may change course after you start working on them. As such, the proposal will not be graded; however, if you do not submit a proposal, it may negatively affect your project grade. There is no specific page limit, but we expect that a 1-2 pages (12-pt single spacing) should suffice. Just make sure to include all the following items:

- Who is on the team? If you are working in a team of two, is there a clear division of labor? If team is smaller/larger than recommended, provide a rationale.
- Describe the problem you are trying to address, and provide a formulation of it. If it involves an algorithm (i.e., any implementation-based project), describe the input to the algorithm and the desired output.
- What is the ideal outcome of the project? What do you expect to show?
- What algorithms do you expect to use?
- What topics / libraries / platforms, if any, will you have to learn in order to undertake your project? Provide references where applicable.
- If your project has a machine learning component, it will most likely involve a dataset. Where will this come from? Is it an existing dataset, or are you making a new one? If the latter, do you have the resources to do so?
- Define a halfway milestone for your project. This will be due on 12/4 – note that this is only a week before your draft report is due. Milestones can include learning about certain topics / algorithms, acquiring and processing datasets, implementing an algorithm, analyzing results, etc. Again, we will not penalize you if you do not achieve your specific milestone. At the due date, we will request a short progress report (see next page) that addresses progress toward the milestones, and if they were not achieved, what turned out to be more challenging than expected. You may also find that your initial milestones were inappropriate, and you can choose different milestones, work toward those, and report on them. That is completely fine too – the point is to decompose your project into smaller pieces and ensure that you are making consistent progress over the next few weeks.
- Provide a week-by-week plan for your project.

Theoretical track

The proposal should be similar to the above, but certain items above may not be relevant (e.g., problem formulation, libraries/platforms, dataset). Instead, provide an initial list of learning material you will study to understand the topic, and provide a list of proposed deliverables that you would like to develop (such as in the form of a report, slides, illustrative examples, or other pedagogical materials).

4 Project Milestone

As previously mentioned, the proposal and the proposed milestone are not treated as contracts, but rather as your best guess of how things will turn out. Therefore, you will not be penalized if you fail to achieve your proposed milestone. In fact, project milestones are rarely reached on time, so we do not expect that of you either.

Why set milestones when they are unlikely to be achieved? It would be nice to accomplish them, so if it does actually work out, good for you. More importantly, as the first module of the course showed, planning is the basis of rational decision making. The milestones you set (and the week-by-week plan requested in the proposal) are supposed to guide your actions, and allow you to determine the feasibility of your proposed project, in terms of the time and effort required to achieve your goals. (Does there actually exist a solution to reach your goal that respects time and resource constraints?) Of course, there is uncertainty in the outcomes, so technically you need to plan under uncertainty ...

... which leads to what we expect for the milestones. So far, we have covered the MDP approach to decision-making under uncertainty – if you were really on top of your project proposal game, you would produce a *policy* instead of a plan. There is another common approach, known as replanning, which we will learn by doing here. This approach is as the name suggests – if the outcome is different from what you expected, plan again from your current state. The point of the milestone deliverables is to reflect on what you have done so far, how it went, and to replan as necessary.

More concretely, we expect milestone reports (one report per team) to address the following 3 'R's (not Reading, wRiting, aRithmetic):

1. **Report**: What have you done so far? What is the current state of the project? To ensure best use of time, we recommend treating this question as a partial draft of the final report – ideally, if your project does not change, you can write 1–2 pages for this section in each milestone report, and then copy and paste them into the final report.

2. **Reflect**: Did you achieve your milestone(s)? If not, why not (e.g., some part was challenging because of [*insert challenge here*] and hence took longer, some other task became irrelevant)? Did you do something extra that you did not mention in your proposed milestone(s)?

3. **Replan**: Based on your progress so far and the knowledge you have acquired, what is the upcoming plan (week-by-week)? What is your immediate next step? If your project milestones / goals have changed from the initial proposal (e.g., you underestimated / overestimated the difficulty of the project), propose new milestones / goals.

Good project progress will roughly follow this timeline (practical track):

- By the milestone (12/4): Have a precise problem formulation (inputs and outputs). Learned the basics of any additional topics necessary to accomplish project. Collected all the necessary external pieces (e.g., cleaned dataset doing machine learning, simulator / model if doing planning or reinforcement learning, external libraries if needed, etc.). High-level description / pseudocode of first algorithm to try, if you haven't tried anything yet; or describe which algorithms you have already tried.

- Stretch goal: Ideally, by the milestone (12/4), you would have already implemented and experimented with at least one algorithm / model. If some algorithm is complete, perform some empirical analysis and identify potential areas of improvement. Also, determine an 'end-game' for the project (i.e., a plan for wrapping up within 1–2 weeks).

- By draft report (12/11): Write up the complete project cycle: What is the high-level problem you wanted to tackle, your precise (technical) problem formulation, the methods / algorithms you used (including external libraries / datasets if any), the empirical results you obtained, some analysis about why the described method worked well / not well in your experiments, some future direction to address what did not work well, possibly propose some interesting extensions, and reflection on the project as a whole (e.g., surprises, challenges, etc.). The report should be relatively complete (it should be a "release candidate", in software engineering terms). See next page for more details.

- By final report (12/18): Any further work and wrapping up, if desired. How much more this contains over draft report depends on how much you worked on the project by that point, and/or how satisfied you are with the outcome(s). If you feel you have put in sufficient time and effort into the project, have achieved the goals you set for yourself, and have a solid written report, then you do not have to do anything beyond the draft report. For others, treat this extra week as an "extension" / second chance – reflecting on our own projects, we find that just before the deadline, when in the rush to get everything working for the presentation / report, we often uncover many more things that we can and want to improve on, as well as loose ends to tie (many TODO comments in the code...). This week is for that purpose – to polish rough, freshly (under-)baked work into a complete, presentable project that you are proud of (and would willingly show to friends / family / potential employers).

5 Draft and Final Report (Practical track)

There is no specific format that the project report should follow. You should choose a structure that best tells the story of your project.

Most reports will probably have these components:

- Describe the problem on a high level, including some motivation for choosing it (just like in your presentation).
- Concrete technical problem statement, model (if applicable), inputs/outputs.
- Describe the dataset you used, if applicable.
- Methods / algorithms that you used or developed. If you are using algorithms not covered in the class, provide a description of it so we know you understand what you have been using.
- If you are basing your project on someone else's work, explain on a coarse level what they have done, and if you are doing anything different.
- Empirical results, including details on the experimental setup (be precise about what settings / data you ran experiments with). What were your hypotheses / what do you expect to see from your experiments?
- Analysis of empirical results. What worked, what did not, and why?
- Discussion / future directions (if any). What difficulties did you encounter, if any? Did anything not go according to plan? If you had more time to spend on the project, what would you have liked to do next? What advice about the project would you give to future CS 4100 students?

Length

There is no set length / page limit for the report. It would probably take at least 6–8 pages (including figures) to include all of the above. You can write as much as you wish, but do not be excessive – just get to the point and provide a complete description of the project, including the components described above.

Draft report

The draft report is due on 12/11, one week before the final report. We *do not* expect the report to be fully written, or even contain all the results. However, we would like to see a paper with the overall structure and sections in place, some filled sections (e.g., if you wrote applicable things in the proposal / milestone reports, you can copy and paste them into the draft), possibly some sections with bullet points, and maybe even some blank or "TODO" sections. The idea is to have a top-down view of the project report (instead of bottom-up based on milestone reports), so we all have a clear picture of what is left to do in the project / write-up. Essentially, the sections that have yet to be filled in the draft becomes your to-do list in the following week while working toward the final report.

6 Final Deliverables (Theoretical track)

This is more fluid and harder to predict. It depends on the chosen topic, its scope, and your team's interests. The default expectation is that you will deliver a 20–30 minute recorded video lecture on your topic, which will be made available to current and future generations of CS 4100 students. Some small-scale implementation of relevant algorithm(s) is also expected, primarily as pedagogical examples to supplement your lecture.

Past theoretical projects have also included some combination of the following (but not all!):

1. A set of lecture notes, written in a tutorial format suitable for self-learning.
2. A set of lecture slides, suitable for teaching fellow CS 4100 students.
3. Illustrative examples/visualizations that aid in understanding concepts/models/algorithms regarding the chosen topic.
4. Simple and well-documented code examples for algorithms studied.
5. Jupyter notebook (or similar tutorial formats) for showing some of the above items.

Suggestions not included in the above list are completely welcome and encouraged – be creative! If you feel strongly about producing a particular theoretical-track project that does not fit the above, please reach out to course staff. Throughout the project, we recommend having more frequent check-ins to jointly define what the final results of the project may look like.

7 Assessment

Since every project is different, there's no single way for us to grade projects, unlike most other assignments. Instead, we look at multiple dimensions that we believe are important abilities as an AI practitioner. There are at least 5 primary dimensions we consider:

1. Formulation: Did you clearly and precisely describe your problem as an AI problem? What AI approach(es) did you consider? Are you able to articulate the approach(es) you used, demonstrating that you understand the underlying techniques and algorithms?

2. Learning: What (if anything) did you have to learn beyond the course material while completing the project? This includes technical material (theory, algorithms, models), implementation (code libraries/frameworks, existing repositories, game engines), domain-related knowledge (rules/strategies of games, dataset interpretation), and more. Tell us what you had to learn and roughly how long it took you.

3. Implementation: What (if anything) did you implement? How involved was it? What parts were existing, and what was your contribution? Although it's unlikely we will actually run your code, including your code (or a GitHub link) provides evidence of implementation.

4. Results: If there was an implementation, what were the outputs/results? How well did your system perform? For theoretical projects, how extensive are your deliverables (quality + quantity)?

5. Analysis: Did the results you obtained and their interpretation make sense? Were there any interesting findings? What further experiments did you perform/consider? If limitations were identified, what steps were taken to address them? For theoretical projects, how extensive are your explanations / comparisons (if applicable)? Is pedagogical intuition provided? Strive for greater depth on the topic you are covering.

6. Other: Size of group, quality of presentation (if required), etc.

Some notes on the above:

- We do not expect you to attain maximum scores on all dimensions – for example, some projects may have more learning involved (e.g., approach not covered in class, theoretical project on a new topic), while others may have more implementation (e.g., using an algorithm covered in class in a new domain). There are many ways to achieve a successful project.

- Based on the draft reports we typically get, the problem/approach(es) can usually be described in greater detail. Show that you understand what you are doing. Sometimes an example helps to illustrate an abstract problem formulation / solution approach.

- We are much more interested in the process (that reflects AI critical thinking) than in the actual final results, especially for practical projects. Many of you probably encountered unexpected challenges along the way. Do not omit these, otherwise we would not know about it! Tell us about the issues you encountered, what steps you took to address them, and why. Again, we want to see the path you took, not just the final destination (reminiscent of search).

- For theoretical projects, aim to make your materials self-contained, suitable for self-study by curious CS 4100 students (or similar). Do not just provide a summary of some topic – teach them (and us). If you are submitting Jupyter notebooks, apart from the `.pynb` files, please save and upload the output as a PDF so we can read it without loading the notebook itself.